

AI Personal Assistant Platform

Multi-Source RAG System

MVP Technical Specification & Development Plan

CONFIDENTIAL

Prepared by	Tarkshy Consultancy Services
Document Type	Technical Specification
Version	1.0
Date	May 22, 2026

1. Product Overview

This document is the complete technical specification for an AI-powered Personal Assistant platform. The system integrates data from multiple sources — Notion, Obsidian, Google Drive, Gmail, Google Calendar, Apple Calendar, and local files — into a unified knowledge layer queried through a sophisticated multi-type Retrieval-Augmented Generation (RAG) pipeline. The MVP is delivered as a cross-platform Tauri desktop application, prioritising offline-first operation, local data storage, and user privacy.

1.1 Core Objectives

- Enable natural-language querying across the user’s entire digital knowledge base.
- Implement six distinct RAG retrieval strategies optimised for different query patterns.
- Build a privacy-first, offline-capable desktop application with fully local storage.
- Minimise operational cost through open-source embedding models and free-tier LLM APIs.
- Ship a production-ready MVP within an eight-week development cycle.

1.2 Scope — Phase 1 MVP

Component	Included in MVP
Data Sources	Notion, Obsidian, Google Drive, Gmail, Google Calendar, Apple Calendar, Local Files
RAG Strategies	Dense, Sparse, Hybrid, Faceted, Contextual, Recursive
Storage	SQLite (documents & metadata) + Qdrant (vectors)
LLM	Groq Inference API (primary) + Ollama (offline fallback)
Embeddings	nomic-embed-text via Ollama — fully local, zero API cost
Frontend	Tauri Desktop App (React + TypeScript)
Authentication	OAuth 2.0 — Google Suite & Notion

2. System Architecture & Technology Stack

2.1 Form Factor — Desktop-First

The primary deployment target is a Tauri desktop application. Tauri uses a lightweight Rust-based runtime combined with a React web frontend, producing a cross-platform binary that runs on Windows, macOS, and Linux from a single codebase. This choice provides native file-system access for Obsidian vault integration, offline-first operation with embedded databases, and a binary footprint of ~50 MB — compared with ~150 MB for Electron alternatives.

2.2 Full Technology Stack

Layer	Technology	Rationale
Frontend Framework	React 18 + TypeScript	Modern, type-safe, component-driven UI development
Desktop Runtime	Tauri 1.4+	Lightweight, offline-capable, cross-platform
UI Component Library	shadcn/ui	Accessible, composable, consistent design system
State — Server	TanStack Query (React Query)	Caching, synchronisation, background refetch
State — Client	Zustand	Minimal global state for UI and user preferences
Build Tool	Vite	Sub-second HMR, optimised production bundles
Backend Runtime	Rust + Tokio	Memory-safe, async, performance-critical ops
SQL Database	SQLite 3	Zero-config, single-file, supports encryption
Vector Database	Qdrant (standalone)	Single binary, <10 ms latency, built-in filtering
Embedding Model	nomic-embed-text (768d)	Open-source, runs locally via Ollama, zero cost
LLM Primary	Groq API — Mixtral 8x7B	Free tier, <500 ms latency, strong reasoning
LLM Fallback	Ollama — Mistral 7B	Fully local, private, unlimited, offline-capable
Document OCR	Tesseract.js	Local PDF and scanned-image text extraction
File Watching	chokidar / Rust notify	Real-time file-system change detection

3. Data Integration Layer

The integration layer is built around a modular manager that normalises data from seven distinct sources into a unified document schema. Each integration implements a common interface exposing fetch, transform, and sync operations, ensuring consistent downstream processing regardless of source.

3.1 Supported Data Sources

Source	Integration Method	Sync Pattern	Data Captured
Notion	Official REST API	Periodic — every 6 h	Pages, databases, properties, nested blocks
Obsidian	File-system watcher	Real-time	Markdown files, YAML front matter, wikilinks
Google Drive	OAuth 2.0 + Drive API v3	Daily + on-demand	Docs (as markdown), Sheets (as CSV), PDFs
Gmail	OAuth 2.0 + Gmail API	Daily + on-demand	Message body, subject, sender/CC, thread ID
Google Calendar	OAuth 2.0 + Calendar API	Real-time listener	Events, descriptions, attendees, timezone
Apple Calendar	CalDAV protocol	Real-time (macOS only)	Events, descriptions, location metadata
Local Files	File-system watcher	Real-time	Markdown, PDF, DOCX, TXT

3.2 Unified Document Schema

All sources are normalised to the following structure before chunking and embedding:

```

id      — "source_uniqueId"    (e.g. "gmail_18a3bc9d")
source  — notion | obsidian | gmail | gdrive | gcal | apple_cal | local
title   — Human-readable document title
content — Full extracted plain-text content
metadata — created, modified, author, url, tags[ ], category
chunks  — Populated by the chunking pipeline (see Section 7)
```

3.3 Per-Source Integration Notes

Notion

Fetches the full page tree recursively via the Notion API. Rich-text blocks are flattened to plain text; database properties are serialised to key-value pairs appended to the document body. Last-edited time is used for incremental sync.

Obsidian

A chokidar file-system watcher monitors the configured vault path. On change, YAML front matter is parsed with js-yaml and wiki-links are resolved to titles. Folders become the document category tag.

Gmail

Query-based sync uses `after:{timestamp} label:{label}` to fetch only new messages. MIME payloads are parsed, HTML stripped, and thread IDs stored to allow future conversation-level RAG retrieval.

Google Calendar & Apple Calendar

Events are stored as documents whose content field is constructed from the description, attendee list, and location. This allows the contextual RAG strategy (Section 4.5) to surface relevant documents when a calendar event is imminent.

4. Multi-Type RAG Retrieval System

Six distinct retrieval strategies are implemented. The system selects the optimal strategy automatically by analysing the incoming query, while also allowing the user to override via a UI control.

4.1 Dense Retrieval

The query is embedded using the same model as the document corpus and the nearest vectors are retrieved by cosine similarity from Qdrant. This strategy excels at conceptual and topical queries where the user may not recall exact wording.

4.2 Sparse Retrieval — BM25

An inverted index (implemented via elasticlunr) applies BM25 term-frequency scoring. This complements dense retrieval for queries containing specific proper nouns, technical terms, or exact phrases not well-represented in the embedding space.

4.3 Hybrid Retrieval (Default)

Dense and sparse result lists are merged using Reciprocal Rank Fusion ($k = 60$). This is the recommended default strategy — it consistently outperforms either method alone across the broadest range of query types.

4.4 Faceted / Filtered Retrieval

Dense or hybrid retrieval is applied first, then filtered by metadata predicates: source type, date range, author, or tag. Essential for users with large, multi-source corpora and for queries that reference a specific source explicitly.

4.5 Contextual / Time-Aware Retrieval

A time-decay function ($\text{score} \times 0.95^{\text{days_since_modified}}$) down-weights stale documents. Documents related to an upcoming calendar event receive a 2× relevance boost. Activated automatically when the query contains temporal keywords.

4.6 Recursive / Multi-Hop Retrieval

The initial result set is sent to the LLM to extract follow-up sub-questions. Each sub-question triggers an additional retrieval pass. Results are merged and de-duplicated before generation. Used for complex queries requiring information from multiple documents.

4.7 Automatic Strategy Selection

Query Signal	Strategy Selected
Temporal keywords (last week, before, since, yesterday)	Contextual — Section 4.5

Query Signal	Strategy Selected
Source reference (in Gmail, from Notion, my Drive)	Faceted — Section 4.4
Complex / multi-sentence query	Recursive — Section 4.6
All other queries (default)	Hybrid — Section 4.3

5. Vector & Relational Databases

5.1 Vector Database — Qdrant

Qdrant is deployed as a standalone single binary with no external service dependencies. It provides sub-10 ms query latency on collections of 100 000+ vectors and exposes a built-in payload filtering API used by the Faceted retrieval strategy. The binary is bundled with the Tauri app and started automatically on launch.

Parameter	Value
Vector dimensions	768 (nomic-embed-text)
Distance metric	Cosine similarity
Indexed payload fields	source, author, created, modified, tags
Storage path	~/.app/vectors/
Startup command	./qdrant --storage-path ./vectors

5.2 Relational Database — SQLite

SQLite provides zero-configuration, single-file persistence for document content, sync state, credentials, and conversation history. Optional AES-256 encryption is available via sqlcipher.

Table	Purpose
documents	Full text, metadata, and sync status for every indexed document
chunks	Individual chunks with foreign key reference to parent document
embeddings	Embedding model name, vector size, and timestamp per chunk
sync_state	Last-sync timestamp and status per integration source
credentials	AES-256 encrypted OAuth access and refresh tokens
chat_messages	Full conversation history for context window construction

6. LLM Integration — Open Models Only

The system supports three LLM providers in priority order. No proprietary closed-cost APIs are required. All inference is routed through a single provider-agnostic client that handles rate-limiting, fallback, and response normalisation.

6.1 Groq Inference API (Primary)

Attribute	Detail
Available models	Mixtral 8x7B (default), Llama 2 70B, Gemma 7B
Inference latency	< 500 ms for 1 000 tokens — fastest available public API
Free tier	120 requests / minute, unlimited monthly volume
Pricing beyond free	USD 0.27 per million input tokens
Default model	mixtral-8x7b-32768
Temperature	0.7
Max response tokens	1 024
Rate limiting	Token-bucket queue implemented in Rust backend

6.2 Ollama — Local Inference (Offline Fallback)

Activated when the Groq rate limit is hit, when the device is offline, or when the user enables the privacy mode setting. Runs entirely on-device with no API calls.

Model	Size	Best For
Mistral 7B	~4 GB	Best quality / speed balance — recommended default
Neural Chat 7B	~4 GB	Optimised for conversational AI tasks
Llama 2 13B	~7 GB	Higher capacity for complex reasoning tasks

6.3 HuggingFace Inference API (Tertiary Fallback)

Used when both Groq and Ollama are unavailable. Free tier, rate-limited. Models: Llama 2, Mistral, Zephyr.

7. Embedding & Document Processing

7.1 Embedding Model

nomic-embed-text is served locally via Ollama and produces 768-dimensional vectors. It delivers quality comparable to OpenAI text-embedding-3-small with zero API cost and no data leaving the device.

Model	Dimensions	Speed	Recommended Use
nomic-embed-text (default)	768	Fast	General purpose — recommended for MVP
bge-base-en-v1.5	768	Medium	Slightly higher quality, same footprint
all-minilm-l6-v2	384	Very fast	Lower memory; good for older hardware

7.2 Chunking Strategy

Three strategies are implemented. The intern should begin with paragraph-based chunking, measure retrieval quality on a test query set, then switch strategies if needed.

Strategy	Description	Chunk Size	When to Use
Paragraph-based(MVP default)	Split at paragraph boundaries; group until token limit	≤ 512 tokens	Best balance of quality and speed
Fixed-size	Hard split every N tokens with configurable overlap	512 / 64 overlap	Uniform documents, quick indexing
Semantic(future phase)	Embedding similarity identifies natural breakpoints	Variable	Highest quality; slow, compute-heavy

7.3 Embedding Pipeline — Step by Step

1. Fetch new or modified documents from the sync queue.
2. Split each document into chunks using the paragraph-based strategy.
3. Estimate token count; ensure no chunk exceeds 512 tokens.
4. Send chunks to Ollama in batches of 32 for embedding.
5. Upsert resulting vectors into Qdrant with full metadata payload.
6. Persist chunk text and references in SQLite.
7. Update sync_state table with the current timestamp.

8. Frontend Architecture — React + Tauri

8.1 Directory Structure

src/components/Chat/	– ChatInterface, ChatMessage, SearchBar, RAGSelector
src/components/Integration/	– NotionSettings, GmailSettings, CalendarSettings
src/components/Documents/	– DocumentBrowser, DocumentPreview, SourceBreakdown
src/components/Settings/	– AppSettings, ModelSettings, StorageSettings
src/hooks/	– useChat, useIntegrations, useDocuments, useRAG
src/stores/	– chatStore, documentStore (Zustand)
src/pages/	– ChatPage, SettingsPage, DocumentsPage

8.2 Key UI Surfaces

Surface	Responsibilities
Chat Interface	Message thread, streaming response display, source citation cards, typing indicator
Search & Query Bar	Natural language input, RAG strategy selector, date-range and source filters
Integration Dashboard	OAuth connect/disconnect, sync status, last-sync timestamps, manual sync trigger
Document Browser	Paginated indexed documents, full-text preview, metadata inspector
Settings Panel	LLM model picker, embedding model, chunk size, sync intervals, privacy toggle

8.3 State Management

TanStack Query manages all server-state: API calls to the Tauri backend, caching, background refetching, and optimistic updates. Zustand holds ephemeral client state: current RAG strategy, active chat thread, and sidebar visibility. User settings are persisted to localStorage for cross-session continuity.

9. Tauri Backend — Rust

All performance-critical operations — document processing, embedding coordination, vector search, LLM routing, and background sync — run in the Rust process using the Tokio async runtime. The React frontend communicates with Rust via Tauri's typed command interface.

9.1 Module Layout

Module	Files	Responsibilities
integrations/	notion.rs, obsidian.rs, gmail.rs, google_drive.rs, calendar.rs	OAuth clients, file watchers, API calls per source
rag/	dense.rs, sparse.rs, hybrid.rs, contextual.rs, recursive.rs	Retrieval strategy implementations, RRF fusion
embedding/	chunker.rs, service.rs	Document chunking, Ollama embedding client
storage/	sqlite.rs, qdrant.rs	Database read/write for both storage layers
llm/	groq.rs, ollama.rs, client.rs	Provider-agnostic LLM client, fallback logic
handlers/	mod.rs	Tauri command handlers invoked from React

9.2 Background Sync Scheduler

A Tokio task spawned at startup runs a continuous sync loop. Each source has a configurable interval; real-time sources (Obsidian, Calendar) use event-driven watchers rather than polling. Failed jobs are re-queued with exponential backoff (1 s → 2 s → 4 s → 8 s, maximum five retries).

10. Indexing & Sync Pipeline

10.1 End-to-End Flow

8. Fetch — Query last-sync timestamp; pull only modified documents from the source API or file system.
9. Transform — Normalise to the unified document schema; extract and clean metadata.
10. Chunk — Split using paragraph-based strategy; enforce 512-token maximum per chunk.
11. Embed — Send chunks to Ollama in batches of 32; receive 768-d float arrays.
12. Store — Upsert vectors into Qdrant; persist chunk text and metadata in SQLite.
13. Commit — Update sync_state with current timestamp; emit progress event to frontend.

10.2 Error Handling

Failure Scenario	Recovery Behaviour
Network unavailable	Operations queued locally; pipeline resumes automatically when connectivity is restored
API rate limit (429)	Exponential backoff per source; Groq fallback to Ollama for LLM calls
Malformed document	Skip and log; proceed with remaining documents; alert user after five consecutive failures
Database write error	Rollback transaction; retry once; surface error to UI if second attempt fails
Embedding timeout	Reduce batch size to 8; retry; log Ollama process state for diagnostics

11. RAG-Augmented Generation

After retrieval, the LLM synthesises a grounded response from the assembled context window. Prompt engineering ensures consistent citation, appropriate tone, and truthful uncertainty signalling.

11.1 Context Window Assembly

Block	Content	Token Budget
System prompt	Role definition, behaviour rules, user profile summary	~300
Retrieved documents	Top-K chunks grouped by source, with title and score	~2 000
Chat history	Last five conversation turns for continuity	~500
Current context	Current timestamp, upcoming calendar event if within 2 hours	~100
User query	Verbatim user input	~100

11.2 System Prompt Design

The system prompt establishes the assistant's identity and constraints. Key directives:

- Ground every answer in the retrieved context; never introduce information from outside it.
- Cite every source by name and type (e.g. "Notion — Project Brief, 14 May 2025").
- If the context is insufficient, say so explicitly rather than speculate.
- Match the user's language register (formal vs. conversational).
- For calendar-related queries, treat upcoming events with higher priority than past ones.

11.3 Generation Parameters

Parameter	Value	Rationale
Temperature	0.7	Balanced — avoids both repetition and hallucination
Max tokens	1 024	Sufficient for detailed answers without verbose padding
Top-p	0.9	Nucleus sampling for coherent output diversity
Stop sequences	["User:", "\n\n---"]	Clean conversation turn boundaries

12. MVP Development Timeline — 8 Weeks

Week	Phase	Deliverables	Exit Criteria
1–2	Foundation	Tauri scaffold, React + shadcn/ui, SQLite schema, Notion + Obsidian integration, Google OAuth flow	Documents fetchable from Notion; Obsidian vault readable; OAuth flow completes
3	Data Processing	Paragraph chunking, Ollama embedding service, Qdrant initialisation, sync worker	100 documents embedded, stored, and retrievable by similarity
4	RAG System	Dense, sparse, hybrid retrieval; faceted filtering; contextual ranking; recursive retrieval	All six strategies return relevant results on a test query set
5	LLM Integration	Groq API client, Ollama fallback, system prompt, context window builder	Coherent, source-cited responses generated for sample queries
6	Frontend Polish	Chat UI, integration settings, document browser, settings panel, error states	All user flows completable; no broken UI states
7	Email & Calendar	Gmail sync, Google Calendar listener, Apple Calendar CalDAV, email threading	Gmail and Calendar data indexed and returned in queries
8	QA & Release	End-to-end tests, latency profiling, performance fixes, documentation	All success criteria in Section 16 pass

13. Configuration & Deployment

13.1 Environment Variables

All secrets are stored in .env.local (git-ignored) and loaded into the Tauri backend at startup.

```
LLM_PROVIDER           = groq
GROQ_API_KEY           = <free key from console.groq.com>
OLLAMA_URL             = http://localhost:11434
EMBEDDING_MODEL        = nomic-embed-text
DATABASE_PATH          = ~/.app/data.db
VECTOR_DB_URL          = http://localhost:6333
NOTION_API_KEY         = <internal integration token>
GOOGLE_OAUTH_CLIENT_ID = <from Google Cloud Console>
GOOGLE_OAUTH_CLIENT_SECRET= <from Google Cloud Console>
```

13.2 Build & Distribution

Platform	Command	Output Artifact
macOS	npm run tauri build	target/release/bundle/dmg/app.dmg
Windows	npm run tauri build	target/release/bundle/msi/app.msi
Linux	npm run tauri build	target/release/bundle/deb/app.deb

14. Technical Decision Matrix

Decision	Selection	Alternative Considered	Reasoning
Desktop runtime	Tauri	Electron	~50 MB vs ~150 MB; Rust backend; native FS access
Vector database	Qdrant standalone	Pinecone cloud	No external dependency; offline-first; single binary
SQL database	SQLite	PostgreSQL	Zero-config; single-file; desktop-appropriate
LLM primary	Groq — Mixtral 8x7B	OpenAI GPT-4o	Free tier; <500 ms latency; no proprietary lock-in
LLM fallback	Ollama local	LM Studio	Headless-capable; OpenAI-compatible API; CLI-friendly
Embedding model	nomic-embed-text	text-embedding-3	Open-source; locally hosted; zero marginal cost
Chunking default	Paragraph-based	Fixed-size	Preserves semantic units; natural breakpoints
RAG default	Hybrid (Dense+BM25)	Dense-only	Consistently outperforms single-strategy on mixed queries
UI framework	React + shadcn/ui	Vue + PrimeVue	Larger ecosystem; TypeScript-native; team familiarity

15. Implementation Guidelines

Area	Guideline
Chunking	Start with paragraph-based. Switch to semantic only if retrieval precision is below 60 % on the test set.
Embedding batching	Use batch_size = 32 by default. Increase to 64 if a discrete GPU is detected via Ollama stats.
Qdrant deployment	Run as a standalone binary, not in Docker. Docker adds 2–3× latency overhead for local queries.
OAuth security	Use PKCE (Proof Key for Code Exchange) for all OAuth flows. Store refresh tokens encrypted via sqlcipher.
Rate limiting	Groq allows 120 requests / min. Implement a token-bucket queue in Rust. High-priority user queries bypass background sync jobs.
Offline behaviour	All search (embedding + retrieval) must work offline. Queue LLM generation requests and process them when the connection is restored.
Latency targets	Embedding < 100 ms · Retrieval < 50 ms · LLM generation < 500 ms · Total end-to-end < 2 s.
Privacy	No document content is sent to any external API without explicit user consent. Embedding vectors are safe to transmit (not reversible to plaintext).
Testing approach	Write integration tests for the full pipeline (ingest → chunk → embed → retrieve → generate) using a fixed 50-document corpus with known ground-truth answers.

16. Success Criteria & MVP Completion

The MVP is considered complete when every item below passes without exception.

#	Criterion	Measurement
1	All integrations connect from UI	OAuth completes; vault path accepted; sync runs without error
2	New documents sync automatically	Document appears in index within the configured sync interval
3	Natural language queries return grounded answers	Manual evaluation on 20 representative test queries
4	Responses cite sources with relevance scores	Every response card displays source name, type, and score
5	System handles 1 000+ documents without degradation	No increase in p95 query latency vs. 100-document baseline
6	End-to-end query latency ≤ 2 seconds	p95 measured across 100 queries on reference hardware
7	Desktop app operates fully offline after initial setup	Wi-Fi disabled; search returns results; LLM queued
8	All six RAG strategies functional and user-selectable	Each strategy returns non-empty, relevant results
9	Error handling — network, rate limits, malformed documents	No unhandled exceptions; UI surfaces meaningful error messages
10	Chat history persists across application restarts	Previous conversation visible on relaunch
11	User settings persist across sessions	Model, interval, and source selections survive restart
12	Zero critical bugs; full logging for diagnostics	No P1/P2 bugs open; structured log output to <code>~/.app/logs/</code>